



Python +

Machine Learning +

Perceptron +

Training a Perceptron Model in Python | John Patrick Roach

9 min read · Sep 24, 2016



John Patrick Roach

Follow



Listen



Share

... More

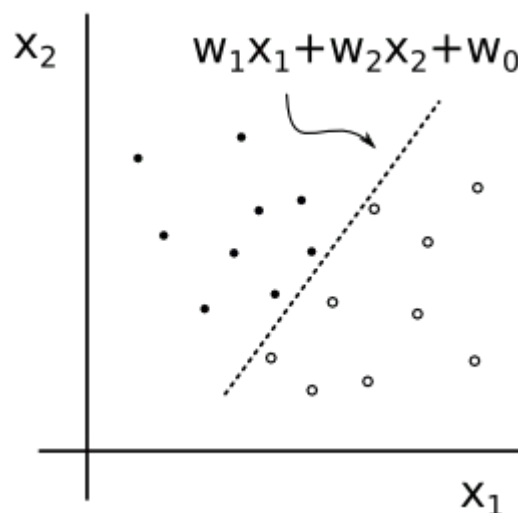
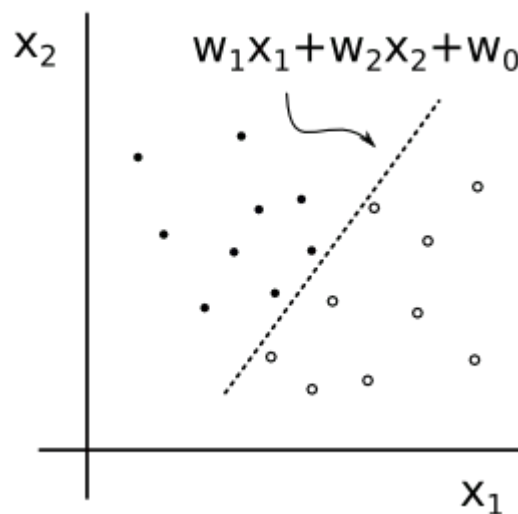
I recently started reading the book *[Python Machine Learning](#)* by Sebastian Raschka. As I go through the book, I plan on doing a series of posts that will outline what I learn along the way. This post outlines the first chapter, “Training Machine Learning Algorithms for Classification”. Specifically, the chapter dives into using a Perceptron model for classification.

Originally based on the artificial replication of neurons firing in brain nerve cells, the linear algebra and algorithms used to quantify how the brain works became some of the earliest beginnings of machine learning. From this the Perceptron model was developed and became a method for supervised binary classification (in the original example, a neuron fires or does not fire). With applications in countless areas, the Perceptron model and machine learning as a whole quickly evolved into one of the most important technologies of our time. As a good historical and study starting point, we attempt to train a Perceptron model in Python here.

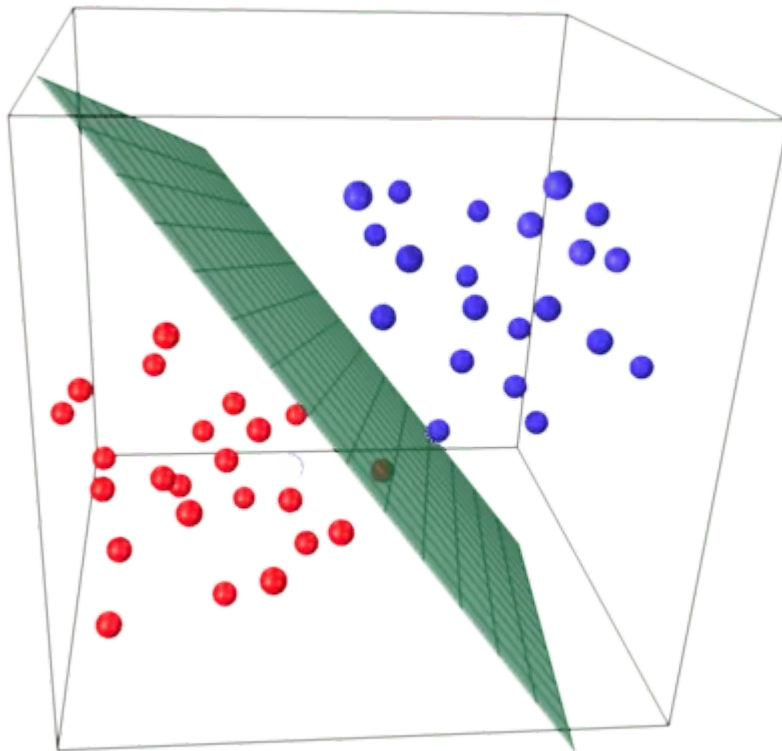
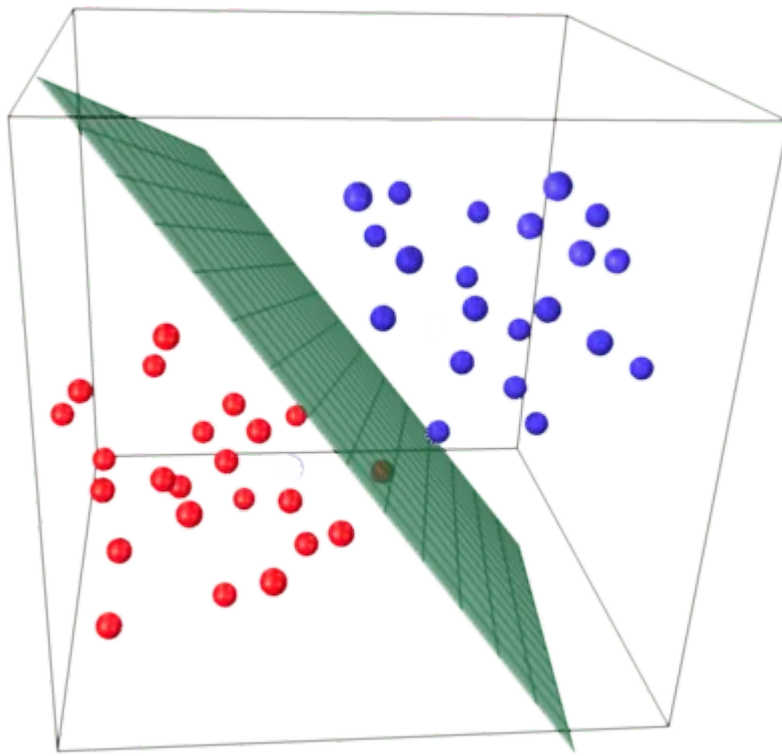
Before we get into the actual Python code and implementation, let's first try to understand the structure of the Perceptron model. It's a supervised binary classification model, supervised meaning we are going to train our model over a set of training data that already has correct outputs, and binary classification meaning our outputs will be two classes. In the neuron example our two output classes were *fire* or *don't fire*. To decide which of the two classes our output chooses, we define an

activation function $\phi(z)$ that takes a linear combination of input values, each with corresponding weight coefficients. Letting z be the net input, we have $z = \dots$. The class prediction is dependent on whether the activation of a particular sample results in an output of $\phi(z)$ that is greater than a defined threshold θ . In the perceptron algorithm, the activation function is a *unit step function*:

If we bring the threshold θ to the left side of the equation and define a weight-zero as $w_0 = -\theta$ and $x_0 = 1$, then we get $z = w_0x_0 + w_1x_1 + \dots + w_mx_m \geq 0$ and $\phi(z) = \{ 1 \text{ if } z \geq 0, -1 \text{ otherwise} \}$. Graphically, if we had two input parameters for each sample (x_1 and x_2), it should look something like this when we are done:



This could easily be extended to higher dimensional data. For example:



In this case, the z is a plane and each sample has three input parameters (dimensions). For dimensions higher than 3, graphical representation becomes more difficult.

The actual training algorithm is very simple.

1. Start the weights at 0 or small random numbers.

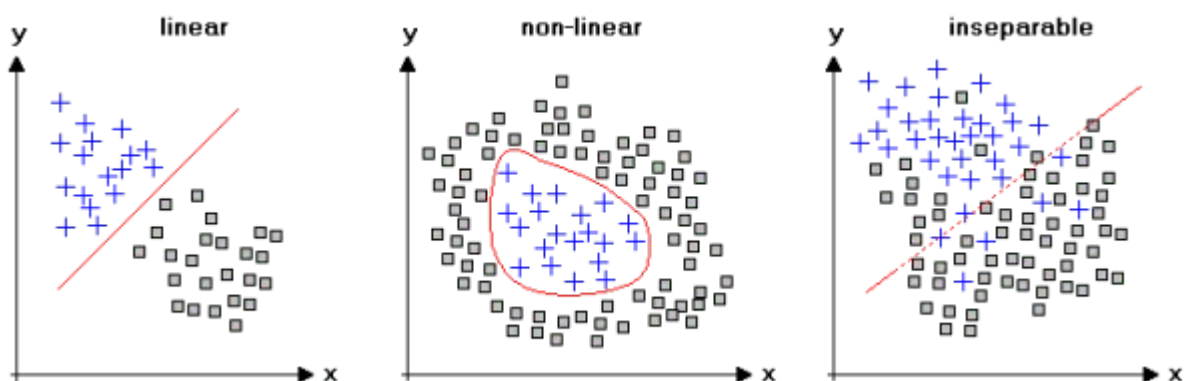
2. For each training sample, compute the predicted output value and adjust the weights if it is wrong.

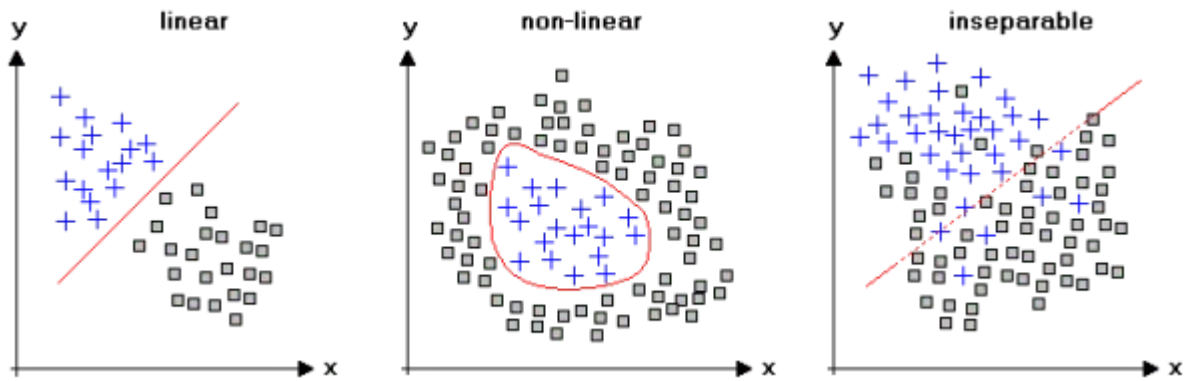
The output value is the class label predicted by the unit step function with our defined threshold included in . Simultaneously, each weight in z is being updated with each training sample, using the underlying learning rule of the perceptron model. Weights are updated as such:

where

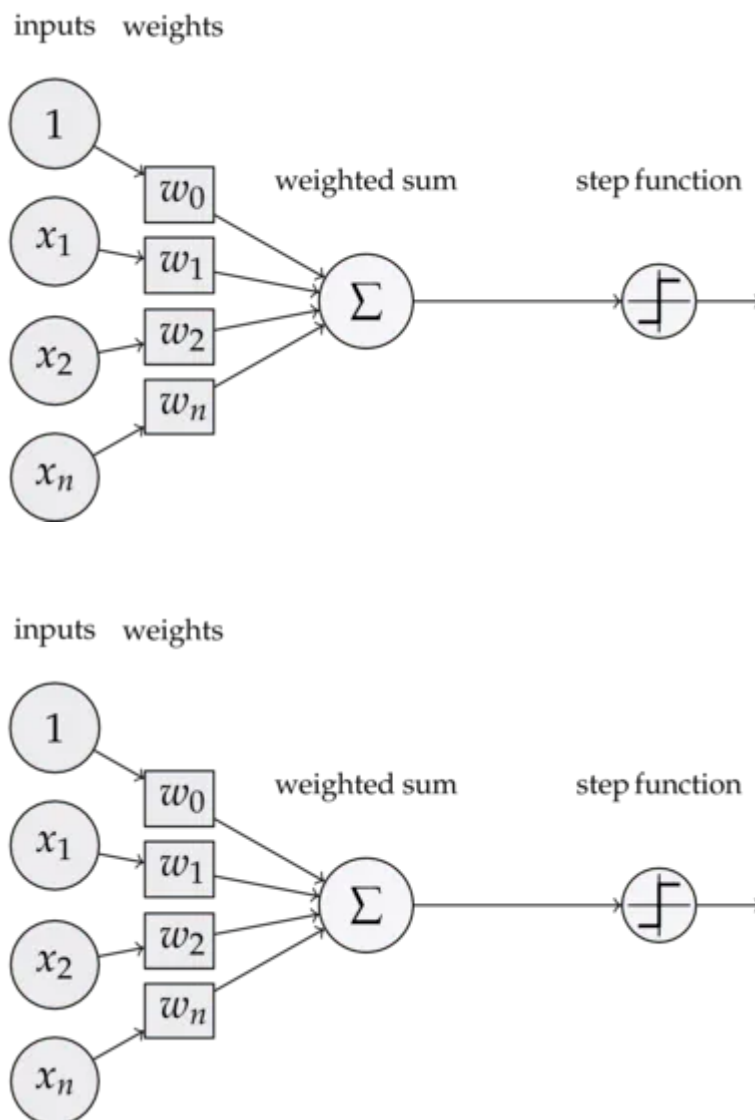
Here, η is the learning rate (a constant between 0.0 and 1.0), y_i is the correct class label for the sample i , and $\hat{y}_i$ is the predicted class label for sample i . Also note that all the weights are updated simultaneously before we recompute z .

Ideally, our perceptron will converge to optimal weight values for predicting correct output classes. Given that our activation take a liner combination input values and that our activation is a unit step function, convergence to optimal weight values requires that our two classes be linearly separable. Otherwise, the perceptron learning rule would never stop updating weights. If the two classes can't be separated by a linear decision boundary, we can either choose a different (non-linear) model, or (if it's close to linearly separable) we can set a maximum number of passes over the training dataset and/or a threshold for the number of tolerated misclassifications. To visualize this:





And, one last visualization of the perceptron learning model looks like this:



Now we are ready to train a perceptron model using Python. The source python code can be downloaded [here](#):

Let's go through some of the code specifics. The code takes an object-oriented approach to define the perceptron interface as a Python `class`. A class allows us to

keep track of the various data parameters with usefully named attributes. The first method defined is the `__init__` method:

```
def __init__(self, eta=0.01, n_iter=10):
    self.eta = eta
    self.n_iter = n_iter
```

The `__init__` method (*method* is a term for functions that are part of a class) is a special Python function that is called when an instance of a `class` is first created. The `self` variable is the instance of the class. The `eta` variable is the learning rate (between 0.0 and 1.0) that we mentioned earlier (η). The `n_iter` variable is the number of passes over the training set that our algorithm will take.

The second method defined is `fit`:

```
def fit(self, X, y):
    self.w_ = np.zeros(1 + X.shape[1])
    self.errors_ = []
    for _ in range(self.n_iter):
        errors = 0
        for xi, target in zip(X, y):
            update = self.eta * (target - self.predict(xi))
            self.w_[1:] += update * xi
            self.w_[0] += update
        errors += int(update != 0.0)
    self.errors_.append(errors)
    return self
```

Here we have three parameters passed in, `self`, `x`, and `y`, where `x` is our array-like training vector with shape = `[n_samples, n_features]`, and `y` is our array-like target values with shape = `[n_samples]`. We also have two attributes `w_` and `errors_`, where `w_` is a 1d-array of the weights after fitting, and `errors_` is a list that will contain the number of misclassifications in every pass over the training data. This method is what our perceptron objects will learn from. In this method we initialize the weights in `self.w_` to a zero-vector using `np.zeros(1 + X.shape[1])`. The 1 we add in this vector is for the zero-weight. This method also loops over all the the

individual samples in the training data and updates the weights according to the perceptron learning rule.

The last two methods are `net_input` and `predict`:



```
def net_input(self, X):  
    return np.dot(X, self.w_[1:]) + self.w_[0]  
  
def predict(self, X):  
    return np.where(self.net_input(X) >= 0.0, 1, -1)
```

The `net_input` calculates the vector dot product of the weight vector and the individual sample vector (what we defined as z earlier), and `predict` is used to predict the class labels (it gets called in the `fit` method).

Now we are ready to start Python and try training a perceptron model on a data set. The data set we will be using is called the Iris dataset, and it contains measurements of 150 iris flowers from three different species: *Setosa*, *Versicolor*, and *Virginica*. In this data set, each flower sample is represented by one row, and the flower measurements in centimeters are stored as columns. Here is the data file:

1. Open terminal and navigate to the folder where you have saved the `Perceptron.py` file. Then run Python from your terminal window once in that folder.
2. Run the following code in Python. This code imports the pandas library, reads our data in as a csv, and displays the last 5 rows of the data with `df.tail()` to ensure the data was read properly. We can see that column 4 contains the true class labels. The first column (0) contains the *sepal length* and the column (2) contains the *petal length*.

```

[proach-nola-m:~ patrick.roach$ cd /Users/patrick.roach/Documents/Python\ Machine\ Learning
[proach-nola-m:Python Machine Learning patrick.roach$ python
Python 2.7.10 (default, Oct 23 2015, 19:19:21)
[GCC 4.2.1 Compatible Apple LLVM 7.0.0 (clang-700.0.59.5)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
[>>> import pandas as pd
[>>> df = pd.read_csv('iris.data', header=None)
[>>> df.tail()
   0    1    2    3    4
145  6.7  3.0  5.2  2.3  Iris-virginica
146  6.3  2.5  5.0  1.9  Iris-virginica
147  6.5  3.0  5.2  2.0  Iris-virginica
148  6.2  3.4  5.4  2.3  Iris-virginica
149  5.9  3.0  5.1  1.8  Iris-virginica

```

```

[proach-nola-m:~ patrick.roach$ cd /Users/patrick.roach/Documents/Python\ Machine\ Learning
[proach-nola-m:Python Machine Learning patrick.roach$ python
Python 2.7.10 (default, Oct 23 2015, 19:19:21)
[GCC 4.2.1 Compatible Apple LLVM 7.0.0 (clang-700.0.59.5)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
[>>> import pandas as pd
[>>> df = pd.read_csv('iris.data', header=None)
[>>> df.tail()
   0    1    2    3    4
145  6.7  3.0  5.2  2.3  Iris-virginica
146  6.3  2.5  5.0  1.9  Iris-virginica
147  6.5  3.0  5.2  2.0  Iris-virginica
148  6.2  3.4  5.4  2.3  Iris-virginica
149  5.9  3.0  5.1  1.8  Iris-virginica

```

1. Next, we import matplotlib.pyplot as plt and import numpy as np. Then we assign the first 100 rows of column 4 (the class labels) to y and use np.where() to set the values of y to -1 where y is 'Iris-setosa', and to 1 otherwise. Finally, we assign the first 100 rows of columns 0 and 2 to X and start to build a scatter plot of this data.

```

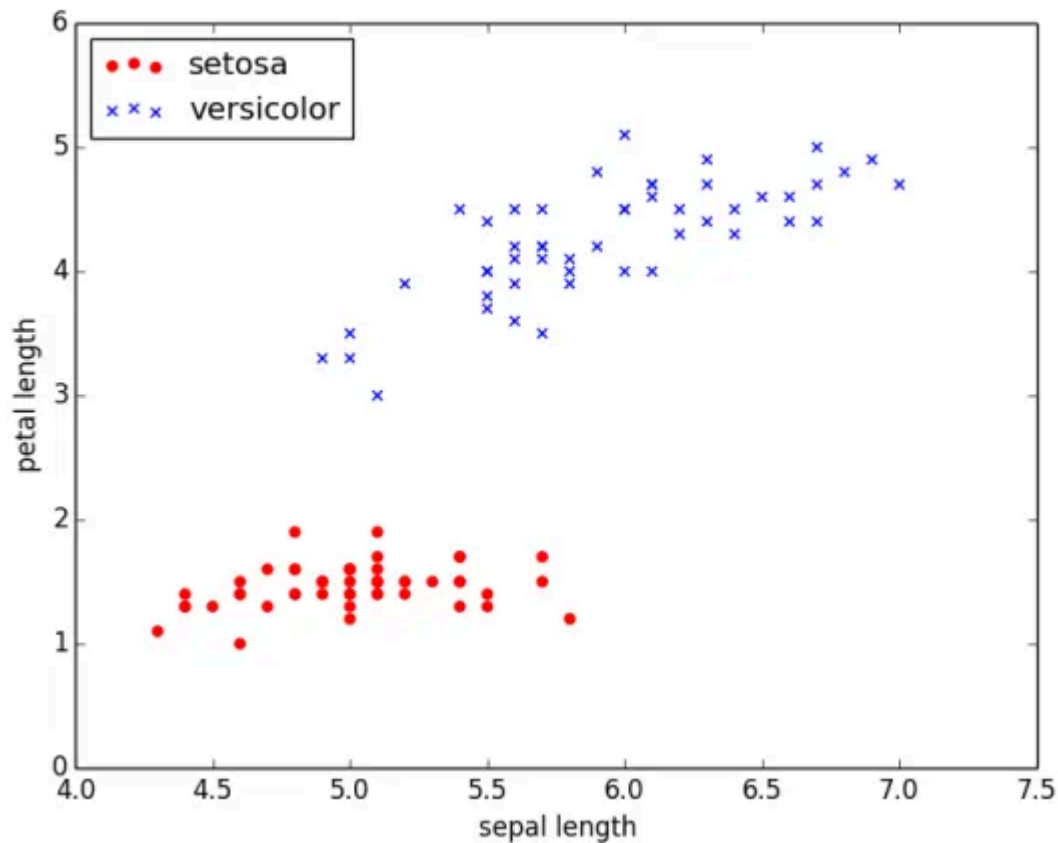
[>>> import matplotlib.pyplot as plt
[>>> import numpy as np
[>>> y = df.iloc[0:100, 4].values
[>>> y = np.where(y == 'Iris-setosa', -1, 1)
[>>> X = df.iloc[0:100, [0, 2]].values
[>>> plt.scatter(X[:50, 0], X[:50, 1],
[...          color='red', marker='o', label='setosa')
<matplotlib.collections.PathCollection object at 0x10d9862d0>
[>>> plt.scatter(X[50:100, 0], X[50:100, 1],
[...          color='blue', marker='x', label='versicolor')
<matplotlib.collections.PathCollection object at 0x10d986a90>
[>>> plt.xlabel('sepal length')
<matplotlib.text.Text object at 0x10a76e510>
[>>> plt.ylabel('petal length')
<matplotlib.text.Text object at 0x10d914050>
[>>> plt.legend(loc='upper left')
<matplotlib.legend.Legend object at 0x10d986750>
[>>> plt.show()
/System/Library/Frameworks/Python.framework/Versions/2.7/Extras
if self._edgecolors == 'face':

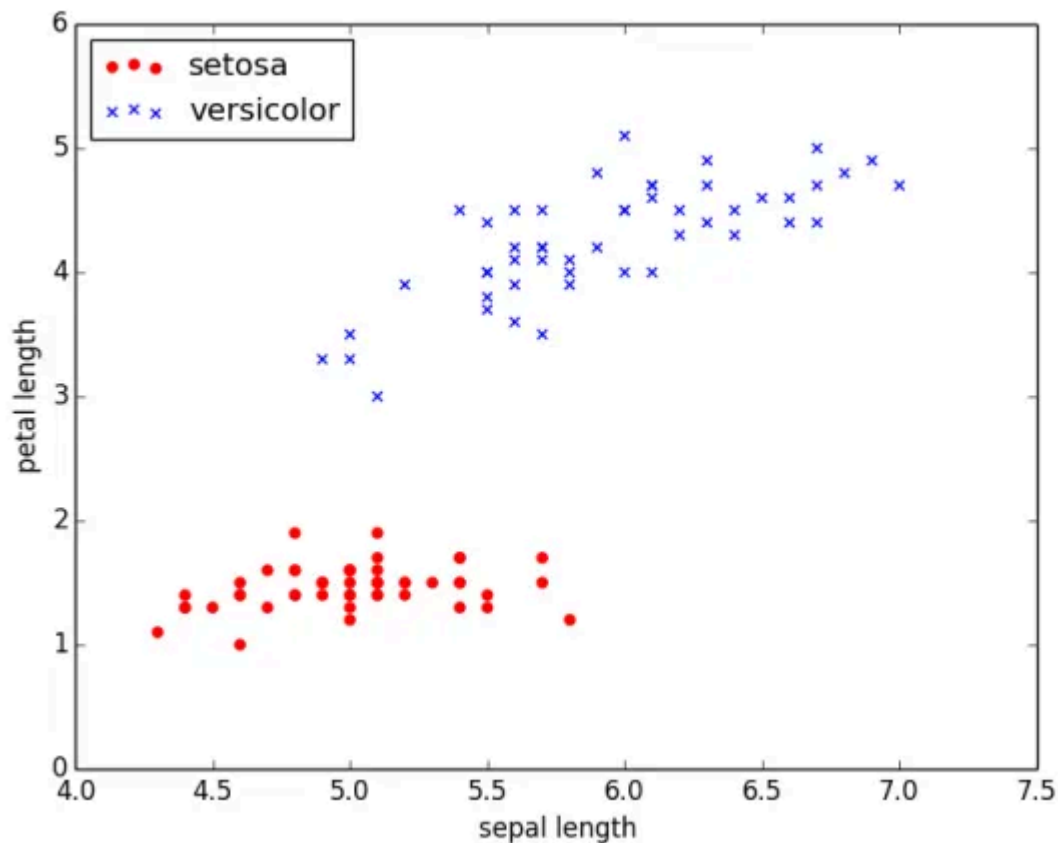
```

```

[>>> import matplotlib.pyplot as plt
[>>> import numpy as np
[>>> y = df.iloc[0:100, 4].values
[>>> y = np.where(y == 'Iris-setosa', -1, 1)
[>>> X = df.iloc[0:100, [0, 2]].values
[>>> plt.scatter(X[:50, 0], X[:50, 1],
[...          color='red', marker='o', label='setosa')
<matplotlib.collections.PathCollection object at 0x10d9862d0>
[>>> plt.scatter(X[50:100, 0], X[50:100, 1],
[...          color='blue', marker='x', label='versicolor')
<matplotlib.collections.PathCollection object at 0x10d986a90>
[>>> plt.xlabel('sepal length')
<matplotlib.text.Text object at 0x10a76e510>
[>>> plt.ylabel('petal length')
<matplotlib.text.Text object at 0x10d914050>
[>>> plt.legend(loc='upper left')
<matplotlib.legend.Legend object at 0x10d986750>
[>>> plt.show()
/System/Library/Frameworks/Python.framework/Versions/2.7/Extras
if self._edgecolors == 'face':

```

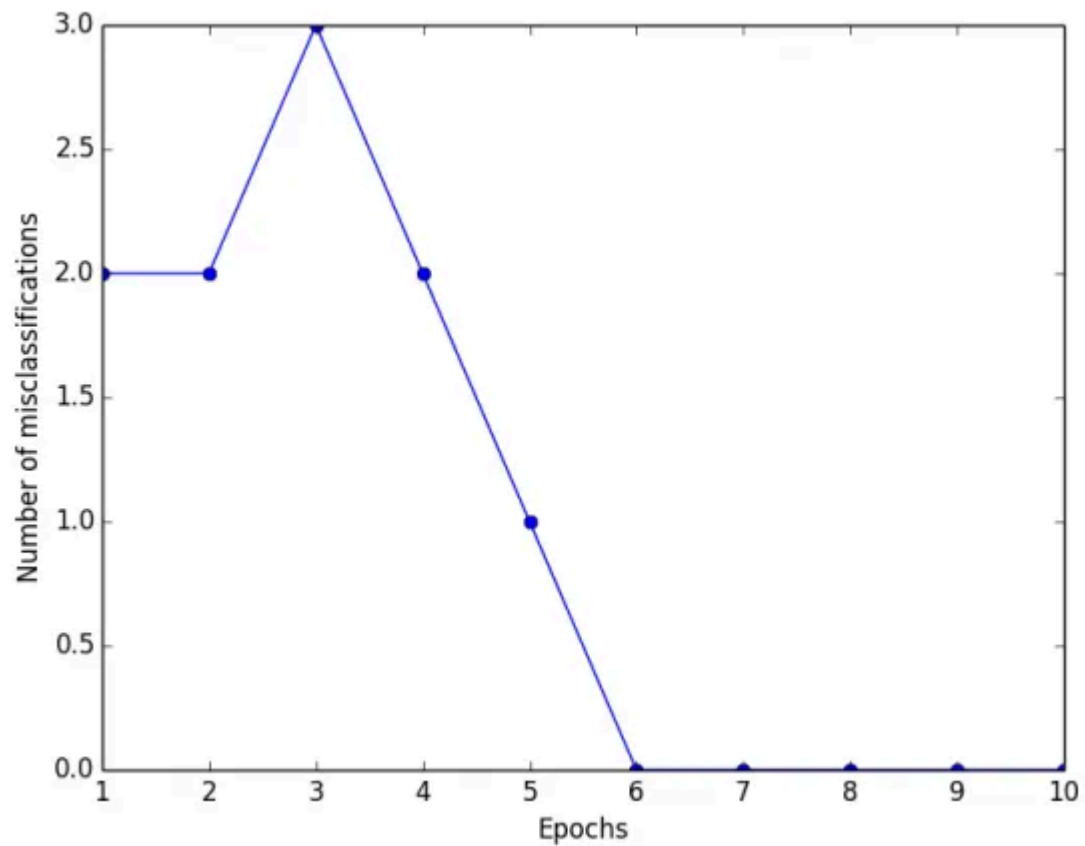


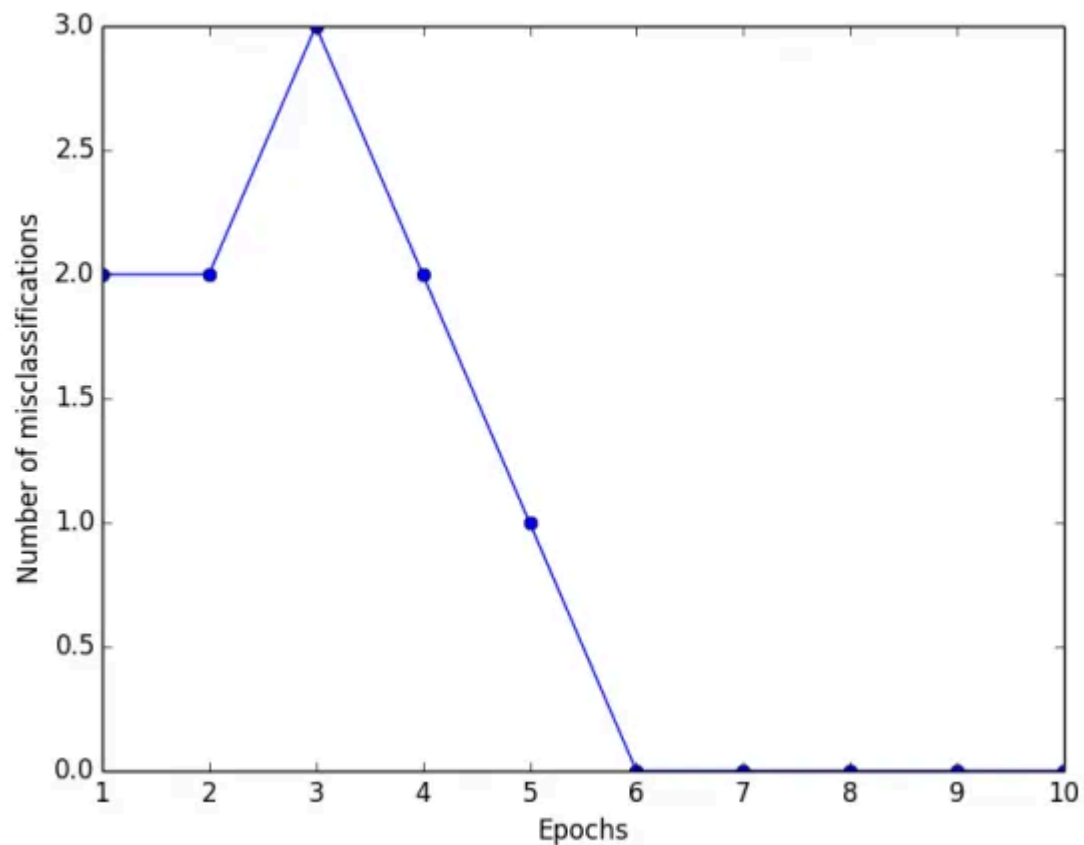


1. We can see from the scatter plot that the two classes may be linearly separable. Now it's time to train the perceptron algorithm on the data subset that we just extracted. We will also verify that the algorithm converges on linear decision boundary by plotting the number of misclassifications for each pass over the dataset. To start, you'll need to import the Perceptron object from the Perceptron.py file you saved. Then we set the learning rate and number of passes over the dataset to execute and implement our fit method over X and y that we just defined.

```
[>>> from ch01 import Perceptron
>>> ppn = Perceptron(eta=0.1, n_iter=10)
>>> ppn.fit(X, y)
<ch01.Perceptron object at 0x10f0f2750>
>>> plt.plot(range(1, len(ppn.errors_) + 1), ppn.errors_, marker='o')
[<matplotlib.lines.Line2D object at 0x10eb06610>]
>>> plt.xlabel('Epochs')
<matplotlib.text.Text object at 0x10d914f90>
>>> plt.ylabel('Number of misclassifications')
<matplotlib.text.Text object at 0x10f12b910>
>>> plt.show()
```

```
[>>> from ch01 import Perceptron
[>>> ppn = Perceptron(eta=0.1, n_iter=10)
[>>> ppn.fit(X, y)
<ch01.Perceptron object at 0x10f0f2750>
[>>> plt.plot(range(1, len(ppn.errors_) + 1), ppn.errors_, marker='o')
[<matplotlib.lines.Line2D object at 0x10eb06610>]
[>>> plt.xlabel('Epochs')
<matplotlib.text.Text object at 0x10d914f90>
[>>> plt.ylabel('Number of misclassifications')
<matplotlib.text.Text object at 0x10f12b910>
[>>> plt.show()
```





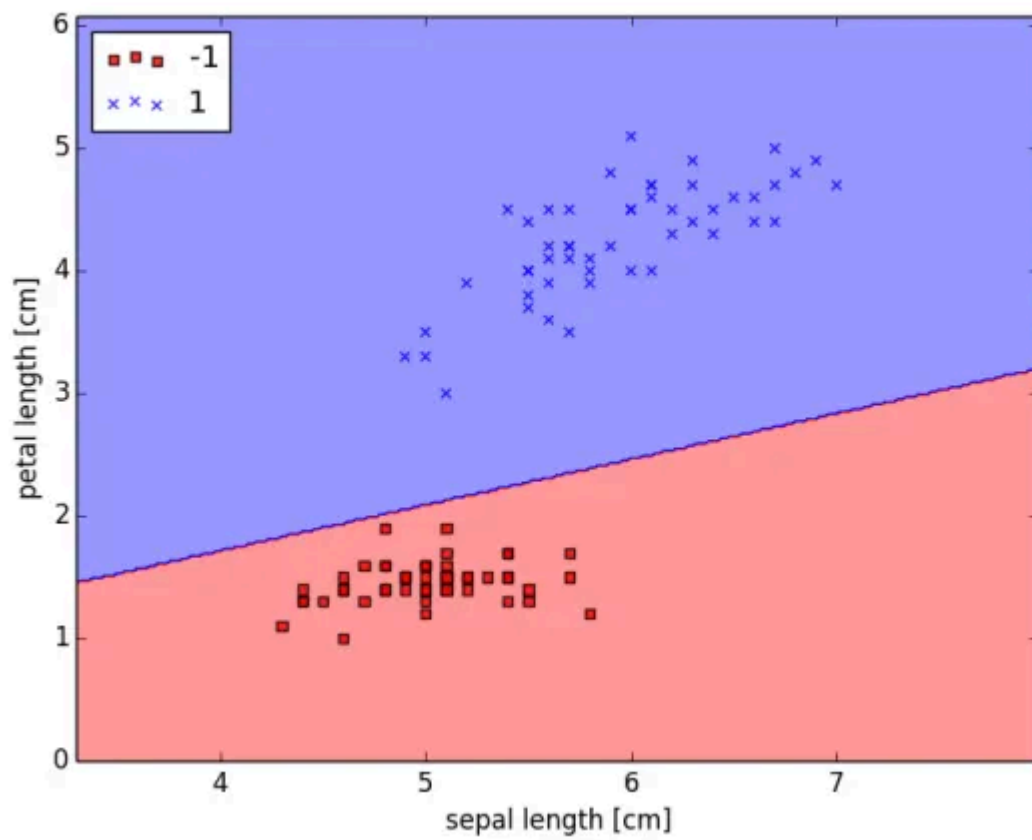
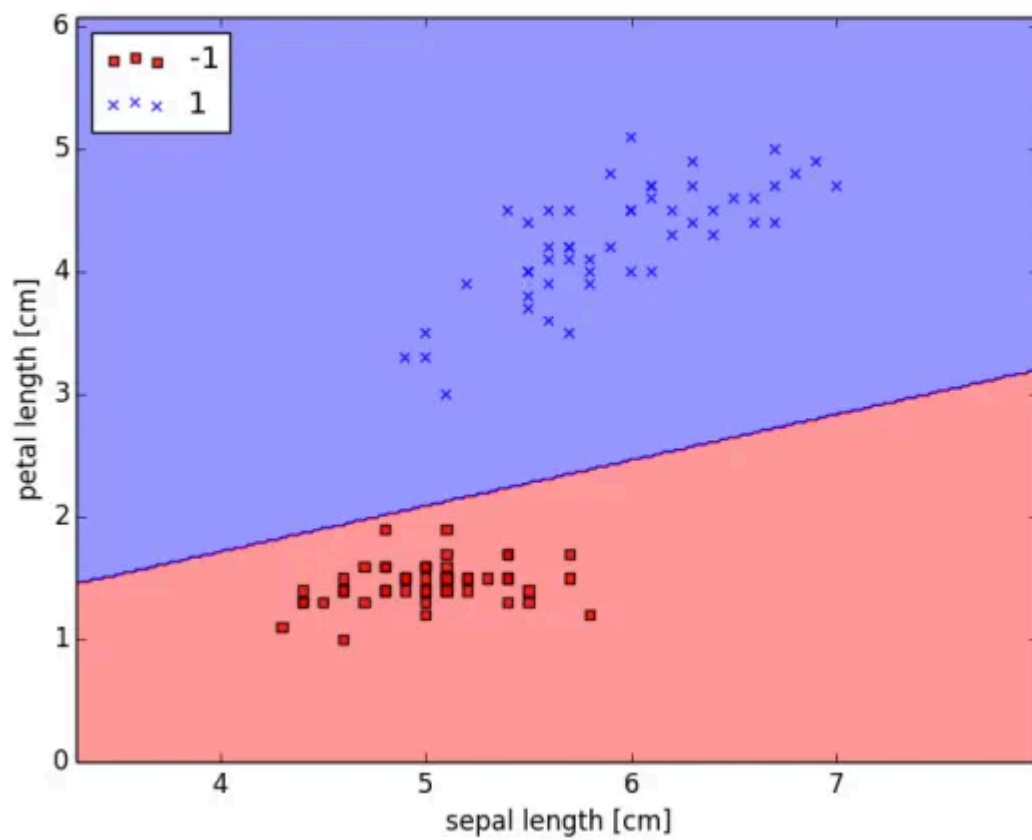
1. We can see from the last plot that on iteration 6 over the data set, our Perceptron algorithm converged on a decision boundary with 0 misclassifications. Now let's plot the decision boundary and our two classes. We'll have to define the plot decision regions and then build the plot.

```
[>>> from matplotlib.colors import ListedColormap
[>>> def plot_decision_regions(X, y, classifier, resolution=0.02):
[...] # setup marker generator and color map
[...] markers = ('s', 'x', 'o', '^', 'v')
[...] colors = ('red', 'blue', 'lightgreen', 'gray', 'cyan')
[...] cmap = ListedColormap(colors[:len(np.unique(y))])
[...] # plot the decision surface
[...] x1_min, x1_max = X[:, 0].min() - 1, X[:, 0].max() + 1
[...] x2_min, x2_max = X[:, 1].min() - 1, X[:, 1].max() + 1
[...] xx1, xx2 = np.meshgrid(np.arange(x1_min, x1_max, resolution),
[...]                        np.arange(x2_min, x2_max, resolution))
[...] Z = classifier.predict(np.array([xx1.ravel(), xx2.ravel()]).T)
[...] Z = Z.reshape(xx1.shape)
[...] plt.contourf(xx1, xx2, Z, alpha=0.4, cmap=cmap)
[...] plt.xlim(xx1.min(), xx1.max())
[...] plt.ylim(xx2.min(), xx2.max())
[...] # plot class samples
[...] for idx, cl in enumerate(np.unique(y)):
[...]     plt.scatter(x=X[y == cl, 0], y=X[y == cl, 1],
[...]                 alpha=0.8, c=cmap(idx),
[...]                 marker=markers[idx], label=cl)
```

```
[>>> from matplotlib.colors import ListedColormap
[>>> def plot_decision_regions(X, y, classifier, resolution=0.02):
[...     # setup marker generator and color map
[...     markers = ('s', 'x', 'o', '^', 'v')
[...     colors = ('red', 'blue', 'lightgreen', 'gray', 'cyan')
[...     cmap = ListedColormap(colors[:len(np.unique(y))])
[...     # plot the decision surface
[...     x1_min, x1_max = X[:, 0].min() - 1, X[:, 0].max() + 1
[...     x2_min, x2_max = X[:, 1].min() - 1, X[:, 1].max() + 1
[...     xx1, xx2 = np.meshgrid(np.arange(x1_min, x1_max, resolution),
[...                             np.arange(x2_min, x2_max, resolution))
[...     Z = classifier.predict(np.array([xx1.ravel(), xx2.ravel()]).T)
[...     Z = Z.reshape(xx1.shape)
[...     plt.contourf(xx1, xx2, Z, alpha=0.4, cmap=cmap)
[...     plt.xlim(xx1.min(), xx1.max())
[...     plt.ylim(xx2.min(), xx2.max())
[...     # plot class samples
[...     for idx, cl in enumerate(np.unique(y)):
[...         plt.scatter(x=X[y == cl, 0], y=X[y == cl, 1],
[...                     alpha=0.8, c=cmap[idx],
[...                     marker=markers[idx], label=cl)
[... 
```

```
[>>> plot_decision_regions(X, y, classifier=ppn)
[>>> plt.xlabel('sepal length [cm]')
<matplotlib.text.Text object at 0x10eab1850>
[>>> plt.ylabel('petal length [cm]')
<matplotlib.text.Text object at 0x10d977350>
[>>> plt.legend(loc='upper left')
<matplotlib.legend.Legend object at 0x10eadd410>
[>>> plt.show()
```

```
[>>> plot_decision_regions(X, y, classifier=ppn)
[>>> plt.xlabel('sepal length [cm]')
<matplotlib.text.Text object at 0x10eab1850>
[>>> plt.ylabel('petal length [cm]')
<matplotlib.text.Text object at 0x10d977350>
[>>> plt.legend(loc='upper left')
<matplotlib.legend.Legend object at 0x10eadd410>
[>>> plt.show()
```



From the plot above it is apparent that our Perceptron algorithm learned a decision boundary that was able to classify all flower samples in our Iris training subset perfectly.

This is just one small example of using a perceptron model to classify data. One of the main things to look out for with a perceptron model is convergence. If the data is not perfectly linearly separable, then you'll need to set a maximum number of passes over the dataset and/or a threshold for the number of tolerated misclassifications.

Now, to most people this dataset on flowers probably isn't that interesting, but it was easily available and already mostly pre-processed for our needs. But, it is easy to imagine other more exciting situations where a supervised classification machine learning algorithm is very useful. For example, imagine a patient comes into the emergency room of a hospital complaining of chest pain and the medical staff's job is to classify the patient as either at high-risk or low-risk of a heart attack. You could use past data on patients where your inputs are data you collected on them through examination and medical tests, and the outputs are whether or not they actually had a heart attack. Using this data, you could train a supervised learning model to classify future patients as at high or low risk of having a heart attack.

Thank you for reading and stay tuned for other learnings on Python machine learning!

-JPR

Originally published at <https://johnpatrickroach.com> on September 24, 2016.

Python +

Machine Learning +

Perceptron +



Follow